

■ 2.2.10 Defining Functions with = and :=

You will have noticed that there are two different assignment operations in *Mathematica*, represented by the = and := operators.

The basic difference between = and := has to do with *when* the expression on the right-hand side of the assignment is evaluated.

$lhs = rhs$ (immediate assignment)	rhs is evaluated when the assignment is made
$lhs := rhs$ (delayed assignment)	rhs is evaluated each time the value of lhs is requested

The two types of assignments in *Mathematica*.

This makes a *delayed* assignment.

Whenever $f[x_]$ is evaluated, the current value of % at that time will be used.

$In[1] := f[x_] := \% + 2x$

Here is an expression.

$In[2] := 1 + y^2$

$Out[2] = 1 + y^2$

This makes an *immediate* assignment. The current value $1 + y^2$ is immediately used, so that the actual assignment done is effectively $g[x_] = 1 + y^2 + 2x$.

$In[3] := g[x_] = \% + 2x$

$Out[3] = 1 + 2x + y^2$

Here is an expression.

$In[4] := 2 + z$

$Out[4] = 2 + z$

f uses the *current* value of %, here $2 + z$. g uses the value of % from the time when the assignment was made.

$In[5] := \{f[a], g[a]\}$

$Out[5] = \{2 + 2a + z, 1 + 2a + y^2\}$

You can see from this example that both = and := can be useful, but they mean different things, and you must be careful about which one you use in a particular case.

One rule of thumb is the following. If you think of an assignment as giving the “value” of an expression, use the = operator. If instead you think of the assignment as giving a “command” for finding the value, use the := operator.

This defines the function `ex` to execute the “command” `Expand`. Every time `ex` is evaluated, the “command” will be re-executed.

```
In[6]:= ex[x_] := Expand[(1 + x)^2]
```

The definition of `ex` maintains the `Expand` “command”, ready for execution.

```
In[7]:= ?ex
ex
ex/: ex[x_] := Expand[(1 + x)^2]
```

`Expand` is called when the function `ex` is evaluated.

```
In[7]:= ex[y + 2]
Out[7]= 9 + 6 y + y^2
```

If you make the original assignment with `=`, then the expansion is carried out immediately.

```
In[8]:= iex[x_] = Expand[(1 + x)^2]
Out[8]= 1 + 2 x + x^2
```

The definition for `iex` now contains the expanded form. The `Expand` “command” is not maintained.

```
In[9]:= ?iex
iex
iex/: iex[x_] = 1 + 2 x + x^2
```

Now the expression `2 + y` is simply substituted into the already expanded form `1 + 2 x + x^2`. The `Expand` command is not re-executed.

```
In[9]:= iex[y + 2]
Out[9]= 1 + 2 (2 + y) + (2 + y)^2
```

$lhs = rhs$	rhs is intended to be the “final value” of lhs (e.g. $f[x_] = 1 - x^2$)
$lhs := rhs$	rhs gives a “command” or “program” to be executed whenever you ask for the value of lhs (e.g. $f[x_] := Expand[1 - x^2]$)

Interpretations of assignments with the `=` and `:=` operators.

You will probably find that you use `:=` more often than `=` in defining functions. If you are in doubt about which to use, it is probably better to try the `:=` operator.

There is nevertheless one common situation where you need to use `=` to define a function. If you do a calculation, and get an answer in terms of a symbolic parameter, say `x`, you will often want to go on and find results for various specific values of `x`. One way to do this, is to take your answer, and make a sequence of replacements for `x` in it. It is usually more convenient, however, to define a *function*,

whose argument gives the value you want to use for x .

Here is an expression involving x .

```
In[10]:= D[Log[Sin[x]]^2, x]
Out[10]= 
$$\frac{2 \cos[x] \log[\sin[x]]}{\sin[x]}$$

```

This defines a *function* whose argument is the value to be taken for x .

```
In[11]:= dlog[x_] = %
Out[11]= 
$$\frac{2 \cos[x] \log[\sin[x]]}{\sin[x]}$$

```

This evaluates the expression taking x to be 2.5.

```
In[12]:= dlog[2.5]
Out[12]= 1.37446
```

Here x is taken to be $1 + a$.

```
In[13]:= dlog[1 + a]
Out[13]= 
$$\frac{2 \cos[1 + a] \log[\sin[1 + a]]}{\sin[1 + a]}$$

```

An important point to notice in this example is that there is nothing “magical” about the name x that appears in the $x_$ pattern. It is just a symbol, and can be part of a result that you got in a completely different computation.